Secure Presence Verification System

Title page

A two-layer attendance verification platform for educational institutions

College / University: **[Enter institution name]**

Department: **[Enter department]**

Submitted by: **[Enter student name(s)]**

Roll number(s): **[Enter roll number(s)]**

Project supervisor: **[Enter supervisor name and designation]**

Academic year / semester: **[Enter academic year and semester]**

Project abbreviation: **SPVS**. Source repository: Secure-Presence-Verification-System-SVPS---Attendance-Software. The inherited repository name uses SVPS; the implemented product and API use SPVS consistently.

Abstract

SPVS is an integrated student attendance system that combines biometric identity evidence with physical-presence evidence. The backend validates a randomized temporal face challenge, a real pretrained face embedding, a device-held signing credential, current GPS measurements and available Wi-Fi observations. It produces explicit present, rejected, review, retry or expired decisions. PostgreSQL transactions and unique constraints prevent duplicate successful attendance, while faculty corrections require attributable reasons.

The implementation uses FastAPI, SQLAlchemy, PostgreSQL, OpenCV SFace/YuNet, MediaPipe, a React institutional dashboard and an Expo student application. It includes actual API integration, migrations, automated tests, model checksums, dependency locks and submission artifacts. Verification evidence is separated from claims that require a physical phone or external attestation configuration. The system demonstrates layered resistance and accountable review without claiming to prevent every spoofing or collusion attack.

1. Introduction

Attendance is both an academic record and a security-sensitive institutional workflow. A presence claim affects eligibility, evaluation and student disputes. A simple token or register makes recording convenient but may provide little evidence about who submitted the claim or where that person was located. At the same time, a complex verification system can harm usability if it silently rejects students or overstates the reliability of noisy sensor data.

SPVS approaches the problem through two primary layers. Identity verification evaluates a consented face template, a fresh challenge and an authorized device. Physical presence verification evaluates location quality, the classroom geofence and supported Wi-Fi signals. Supporting authentication, academic configuration, reports and auditing make those layers usable in an actual college workflow.

2. Problem statement

Traditional methods allow shared tokens, proxy responses, remote submissions and unaccountable edits. A mobile client is an untrusted input source; it must not declare its own face similarity or distance from a classroom. Shared tokens, expired sessions and repeated requests require server-side controls.

Software-only verification cannot absolutely prove presence against a fully compromised phone or coordinated attackers. The engineering problem is therefore to enforce meaningful gates, expose uncertainty, preserve accountability and evaluate the limitations honestly. SPVS excludes surveillance cameras and additional beacon hardware, keeping the project reproducible with normal student phones.

3. Objectives

1. Implement a complete role-based academic attendance workflow.
2. Use actual model-generated face embeddings with consent and protected template storage.
3. Validate randomized temporal challenges on the server.
4. Bind evidence to an authorized device credential and a fresh session.
5. Calculate geofence and signal decisions from submitted measurements on the server.
6. Make retries, replay, concurrency and corrections deterministic and testable.
7. Provide understandable student outcomes and an accountable faculty review process.

8. Deliver source, documentation, measured test evidence and reproducible setup.

4. Existing systems and limitations

Manual registers are simple but allow proxy marks and require transcription. Static QR or code-based systems are convenient but shared tokens can travel beyond the classroom. Pure GPS systems are affected by indoor uncertainty and spoofing. Face-only systems may accept photograph/video presentation if identity matching is confused with liveness. Device identifiers can be copied and do not establish cryptographic possession.

Adding more signals does not automatically establish strong assurance. A high weighted score can conceal a failed security gate if the decision algorithm is poorly designed. SPVS instead enforces mandatory identity/session/device conditions before evaluating supporting location evidence, then routes ambiguity to review or retry.

5. Proposed system

The student receives an institution-managed account, registers one device and consents to biometric enrollment. Faculty open a session for a course and configured classroom. The student requests a bound challenge, performs a short face sequence and collects real foreground location and supported Wi-Fi data. The app signs exact evidence bytes and submits them to FastAPI.

The server checks access, enrollment, session timing, device signature and challenge ownership. It detects and aligns the face, calculates SFace embeddings, compares each frame and analyzes temporal landmarks. Only after these gates pass does it evaluate geofence uncertainty, freshness, Wi-Fi policy and anomalies. A successful decision inserts one attendance record; other states remain attempts that can be disputed and reviewed.

6. Related work and implementation references

The system builds on documented OpenCV face detection/recognition APIs, MediaPipe facial landmarks, modern password hashing and relational transaction controls. OpenCV's published face tutorial supplies an example matching baseline, not an experimental result from this project. Android documentation explains why Wi-Fi scanning and location permissions cannot be assumed to work uniformly. Google and Apple attestation guides describe provider verification boundaries that require external configuration.

The bibliography lists official documentation, pinned model source and actual test-image provenance. No invented publication, benchmark accuracy, institutional deployment or user study is claimed.

7. System requirements

The backend requires Python 3.11/3.12-compatible dependencies, PostgreSQL 15+ (16 in Compose), the pinned model files and generated JWT/Fernet secrets. The dashboard requires a supported Node runtime and a modern browser. Student verification requires a camera-equipped Android/iOS phone, foreground location permission and a reachable API. Android native testing needs a development build, compatible JDK/SDK and the included local Wi-Fi module. iOS requires macOS/Xcode signing and exposes no general Wi-Fi scan API in this implementation.

The primary developer target is a MacBook Air M4. Native ARM64 Python/Node and Docker Desktop are documented. Automated execution in this submission occurred on Linux x86_64; that does not establish Apple Silicon runtime testing.

8. Architecture

The architecture has client, API/service and persistence tiers. The dashboard performs staff operations through a same-origin API proxy. The student app holds credentials in OS secure storage and sends signed evidence. FastAPI enforces roles and scopes, coordinates verification services and records outcomes in PostgreSQL.

The architectural, sequence and decision diagrams in `SYSTEM_ARCHITECTURE.md` are part of this report's generated PDF appendices. OpenCV and MediaPipe inference are synchronous; FastAPI synchronous handlers use its worker thread pool. A per-worker lock guards model instances. Database rate limits and transactional locks remain shared across API workers.

9. Database design

Twenty normalized application tables cover accounts/subtypes, courses/enrollment, classrooms/Wi-Fi, policy, devices/resets, templates, sessions/challenges, attempts/records, events/corrections/disputes, refresh tokens and rate buckets. Foreign keys preserve ownership and attribution. Unique student/session records prevent duplicate official attendance. A partial unique active-device index enforces the normal one-device limit.

Attempts retain reasons and a coarse evidence summary rather than raw images. Corrections retain the actor, reason and previous/new status. Refresh tokens are hashed and grouped into revocable families. The explicit Alembic migration and ER diagram are included in the database appendix.

10. Implementation methodology

Development proceeded from relational/authentication foundations to academic workflows, identity/presence services, client integration, tests and documentation. Each API operation uses a Pydantic input contract. Unknown fields, invalid coordinates and malformed timestamps are rejected. Role-scoped queries and object checks protect backend data independently of UI navigation.

The dashboard provides labeled forms, searchable tables, status labels, real aggregate charts and scoped exports. The mobile app provides enrollment consent, permission handling, sequential camera capture, location acquisition, signing and result/history screens. Android reads recent real scan results; iOS reads only permitted current-network BSSID. Both report unavailable evidence explicitly.

11. Identity verification design

YuNet detects exactly one usable face; SFace generates a normalized 128-dimensional embedding from the aligned crop. The enrolled template is encrypted under a key separate from JWT signing. Every frame must match the reference at the configured threshold. Enrollment frames also match one another before averaging.

MediaPipe supplies facial landmarks. A blink requires open/closed/open eye-aspect transitions. Head actions use normalized nose displacement and stable consecutive observations. Frames must be ordered, within the challenge period and separated by plausible time intervals. Challenge IDs are unpredictable and single-use. These controls provide basic replay resistance, not certified capture provenance or comprehensive deepfake resistance.

12. Physical presence design

Haversine distance is calculated on the server. Location age, future timestamps, uncertainty and reported mock indicators are checked before accepting a geofence. An uncertainty circle fully outside the radius rejects; an overlapping boundary routes to review. Android Wi-Fi observations are matched by known BSSID and minimum RSSI, requiring at least half of configured references when available.

An explicit Android policy controls missing Wi-Fi: review, reject or reduced-assurance GPS-only. The separate iOS policy defaults to review, with explicit retry/GPS-only options. A coarse travel rule flags implausible movement over short periods. Unconfigured platform attestation never returns a verified result. Numerical risk communicates concern and never bypasses a failed mandatory gate.

13. Security design

Passwords use Argon2id. JWTs validate issuer, audience, expiry and token type. Refresh sessions rotate transactionally and revoke their family after reuse. Device signatures cover both an idempotency key and exact JSON bytes, using a native P-256 private key and registered server-side public key (legacy HMAC remains readable for migration).

Student/challenge/session locks protect the attendance transaction. A retry returns the existing attempt only when the signed evidence matches. A changed payload conflicts, and a consumed challenge cannot be reused with a new key. A faculty correction uses the same session lock and produces a separate audit trail. Input body/decoded-image bounds and shared rate buckets constrain abuse.

14. Testing and validation

The project includes unit, database integration, client interaction, browser and real-model smoke tests. PostgreSQL-backed tests run the migration and verify role restrictions, CRUD, refresh replay, device mismatch/revocation, consent, session expiry, duplicates and concurrent retries. Rule tests cover location and temporal sequences. Browser tests interact with the actual running API and database. Mobile logic tests check signing interoperability and decision wording; Metro/Hermes export checks bundle construction.

The API happy-path tests explicitly replace vision to isolate transaction behavior. Real-model tests separately compare public portraits and reject zero/multiple faces. Neither proves full live-phone verification. Current commands, counts and outcomes are included from `TEST_RESULTS.md` in the generated report. Failed intermediate runs were investigated and are not represented as passing results.

15. Actual results

The source contains functional backend, dashboard and student app implementations rather than generated placeholders. Authentication, academic configuration, sessions, reports and audits have been exercised against PostgreSQL. Real pretrained face-model inference has been executed. Verification state handling is connected to the API and constrained database writes. The dashboard builds and browser workflows connect to actual endpoints; mobile logic and bundle checks are recorded separately.

The final result table is generated from executed test result files. No population biometric accuracy, real-device liveness success rate, production latency, enrollment count or deployment adoption is reported because those studies were not performed. Native build status, remaining upstream dependency advisories and provider limitations appear in the verification appendix.

16. Limitations

The system cannot guarantee presence against compromised clients or coordinated credential sharing. Basic landmark liveness can be attacked, GPS has indoor uncertainty, Wi-Fi is platform-dependent and attestation is unconfigured. Thresholds need institution-specific calibration and a consented evaluation dataset. Access tokens have an expiry window after logout; audit storage is not externally immutable. Large-class performance, retention automation, key rotation and fully managed production deployment require further engineering.

The current Expo dependency tree has unresolved upstream advisories recorded in the limitations appendix. These must remain visible in a college evaluation and future release decision. Real-device and Apple Silicon checks remain separate from cloud test results.

17. Future enhancements

Priorities are provider-backed integrity validation, stronger capture provenance/presentation-attack detection, accessible alternative attendance workflows, calibrated thresholds, measured false-accept/false-reject behavior, pagination, operational retention and protected external audit export. Future work should be based on observed weaknesses and institutional requirements. Excluded beacon, ultrasound, surveillance and blockchain hardware/systems are not proposed as hidden dependencies of this implementation.

18. Conclusion

SPVS demonstrates how identity and location evidence can be combined with server-side enforcement and attributable review in an actual attendance application. The strongest implemented properties are explicit decision gates, relational consistency, evidence-bound retries and accountable corrections. Its practical value depends on honest handling of uncertainty and measured device behavior rather than an absolute anti-proxy claim.

19. References

The full bibliography is in `REFERENCES.md` and is included in the report appendix. It contains official framework, model, operating-system and security documentation plus the actual public test-image sources and relevant unresolved advisories.

20. Appendices

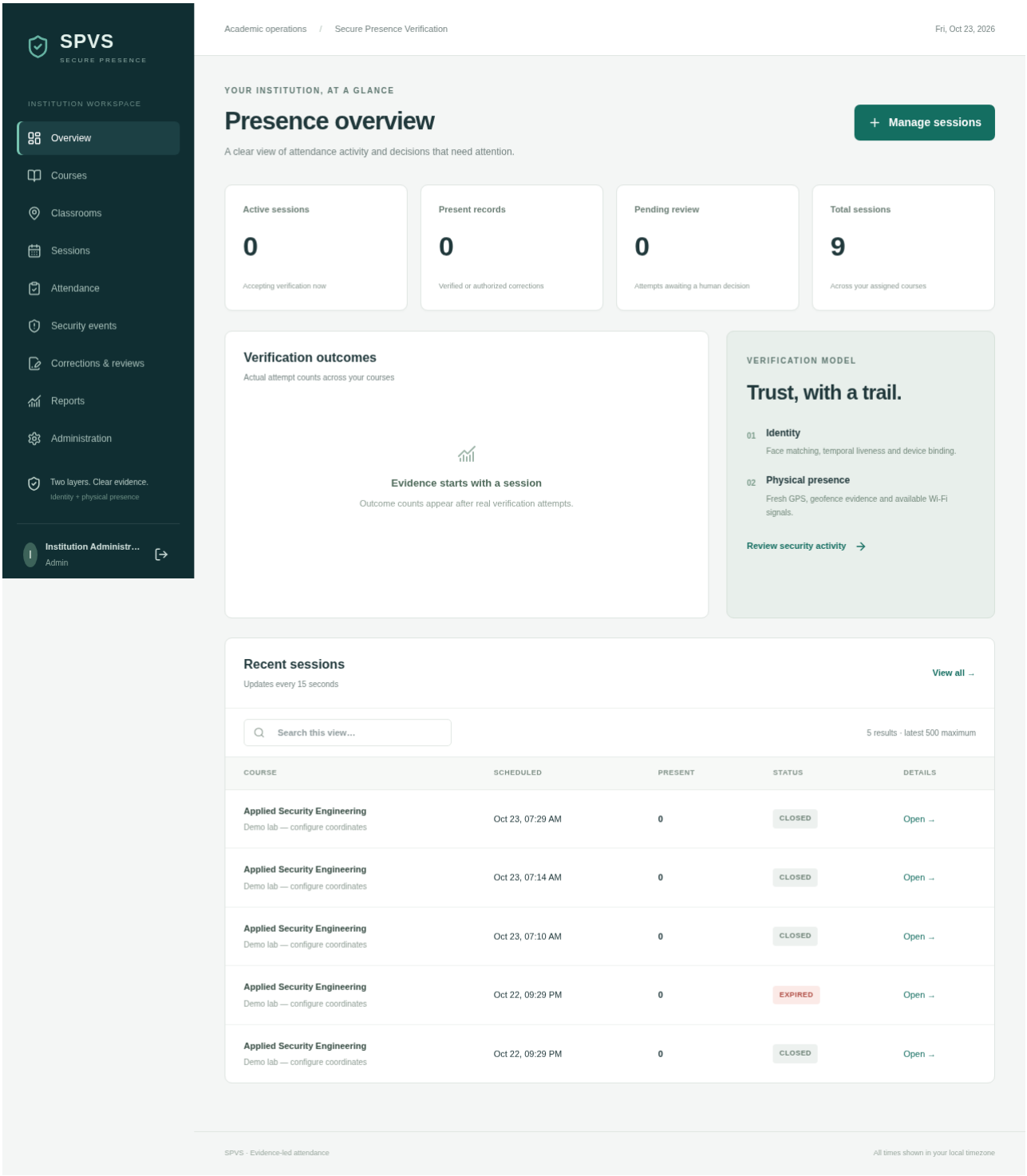
The generated PDF appends architecture/ER/sequence diagrams, the API map, biometric and security design, threat model, current test results, requirements traceability and known limitations. Editable source Markdown and the packaging script are included in the ZIP so that the student can add genuine institutional measurements and academic details without rebuilding the software from scratch.

Cross-platform completion update

This continuation adds native Android Keystore/iOS Secure Enclave P-256 signing, nonce-based device registration, Apple App Attest verification architecture, current-network-only iOS Wi-Fi evidence, precise-location handling and a separate audited iPhone presence policy. Shared enrollment/liveness/session screens continue to use the real backend model pipeline. Automated evidence distinguishes real-model smoke tests from synthetic API/certificate tests, native compilation from JS exports, and unperformed physical-device validation. The appended cross-platform chapter and final verification report are authoritative for release outcomes; no biometric accuracy or provider-backed success has been invented.

Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, even if the global attestation requirement is off. The registered platform is client-reported; without this gate an unverified client could select a weaker platform policy. With App Attest unavailable/unconfigured, iPhone verification remains review/retry. GPS-only evidence still has lower presence assurance after app verification.

Application evidence · actual dashboard



Actual browser screenshot from the evaluated workspace. Counts reflect real demo activity; no attendance was fabricated.

Cross-platform implementation

This completion extends commit `b19777df3cb3210330075c702e5e98f487213a5d`; FastAPI/PostgreSQL, React dashboard and Expo/React Native remain the architecture. The source supports Android and iOS. See FINAL_COMPLETION_REPORT.md for observed builds and tests; source implementation is not physical-device validation.

Shared application and platform adapters

Authentication, enrollment and deletion/re-enrollment, session selection, randomized camera capture, result, history and API recovery use the same screens. `src/platform` owns native device keys, OS selection, Wi-Fi APIs and simulator API defaults. `capture.ts` executes the server's action order and expiration, cancels on backgrounding, and records capture-start timestamps. It does not calculate a successful biometric verdict on the client. `signals.ts` requests foreground location, checks precise accuracy where available and submits the actual fix time and horizontal uncertainty. Failed requests display recovery instructions. Camera temporary files are deleted in finally blocks.

Signal	Android	iOS	Server treatment
Face images	Expo Camera front camera	Expo Camera front camera	Same YuNet/SFace and MediaPipe inference
Liveness	JPEG still sequence, randomized prompts	Same protocol	Sequence, timing, duplicate-frame and identity gates
Device signature	Android Keystore P-256	Secure Enclave P-256 on physical devices; Keychain on simulator	ECDSA SHA-256, DER signature, registered SPKI key
Nearby Wi-Fi	Recent OS scan cache, measured BSSID/RSSI	Not available to ordinary apps	Never synthesized
Current Wi-Fi	Scan evidence	NEHotspotNetwork.fetchCurrent, BSSID only	Separate current-network signal; no invented RSSI
Location	Expo location provider; coarse/full permission	Core Location through Expo and native accuracyAuthorization	Freshness, uncertainty, geofence, mock flag, travel checks
App attestation	Play Integrity remains unconfigured	App Attest native API plus server verifier	Unverified evidence is never reported as verified

Permissions and reproducible projects

`app.config.js` and `plugins/withSpvs.js` configure bundle/package `edu.spvs.student`, display name SPVS Student, portrait orientation, icons, launch screen, iOS 16.4 minimum, camera/location/local-network descriptions, Access Wi-Fi Information entitlement and Android backup exclusion. Set `SPVS_APP_ATTEST_ENV=development` or `production` when generating an App Attest build. The entitlement is deliberately absent without that explicit setting. No background location, microphone or unrestricted iOS Wi-Fi scan is requested. Android location permission gates scan access; scan throttling and OEM restrictions can return unavailable.

Run `npm ci` then `npx expo prebuild --no-install`. Generated Android and Xcode projects are derived artifacts, excluded from Git and the source ZIP. Local Swift/Kotlin Expo modules and podspecs are included. `python3 scripts/verify_native_config.py` checks generated files, not native compilation. The CocoaPods Gemfile pins 1.16.2; run `bundle install` before `bundle exec pod install`. A Podfile.lock must be reviewed and retained after the first genuine macOS installation; this Linux run cannot produce a validated CocoaPods resolution.

Release transport requires an HTTPS API endpoint. For an isolated local development network only, set `SPVS_DEV_HTTP=1` before prebuild and use a development client. Production configuration defaults to TLS and disables Android cleartext. Do not use local HTTP for real student data. Android emulator defaults to 10.0.2.2; iOS simulator defaults to localhost. Set EXPO_PUBLIC_API_URL explicitly for physical phones and release builds.

Device binding and lifecycle

Registration requests a short-lived server nonce. The client signs `SPVS-REGISTER-v1.<nonce>.<platform>.<SHA256(SPKI bytes)>` using its newly generated P-256 key; the server verifies password confirmation, ownership, expiry, signature and the single-active-device constraint. Evidence signs the exact UTF-8 bytes `<idempotency_key>.<payload>`; the signature is base64 DER ECDSA/SHA-256. Neither platform exports a private key to JavaScript. The iOS physical-device path does not silently downgrade from Secure Enclave when key generation fails. Simulator Keychain keys have lower assurance.

A storage label is a client report, not proof of hardware protection. The server exposes this distinction in its admin device list. Old HMAC registrations continue to work for migration; new shared-secret registrations are disabled by default (`ALLOW_LEGACY_DEVICE_REGISTRATION=false`). Test fixtures explicitly enable legacy compatibility. Retire old devices through the audited reset process.

Native installation IDs and stored key aliases detect reinstall/key loss on submission. Signing never silently creates a replacement key. A lost/revoked credential requires an administrator-approved reset. iOS Keychain items can survive uninstall; an installation ID mismatch prevents their silent reuse. The existing device remains authorized until reset approval. Each student's local metadata is stored separately, device-only, in SecureStore. Native storage compromise, rooted devices or injected clients are not solved by a client-reported storage label.

Apple App Attest setup and protocol

1. In the Apple Developer portal register the exact bundle ID and enable App Attest and Access Wi-Fi Information. Use a matching development/distribution provisioning profile. App Attest requires a supported physical device; simulators return unsupported.

2. Retrieve Apple's **App Attestation Root CA** from Apple's official certificate authority page over verified HTTPS. Independently verify its SHA-256 fingerprint through a trusted institutional setup process. Do not accept a root supplied by a client or a certificate from an arbitrary blog. Store the PEM outside the repository.

3. Configure the backend with `APPLE_APP_ID=<TeamID>.edu.spvs.student`, `APPLE_ATTEST_ENVIRONMENT=development` (or production), `APPLE_ATTEST_ROOT_PEM=/run/config/apple-app-attestation-root.pem` and `APPLE_ATTEST_ROOT_SHA256=<verified hex fingerprint>`. The certificate is public, but these are deployment trust settings. Mount the PEM read-only in Docker. All four values must be set; production/development separation is enforced.

4. Set the corresponding `SPVS_APP_ATTEST_ENV` before prebuild or use the EAS profile, sign and install the development build. Register the SPVS device, then choose Register Apple app attestation.

5. `/attestation/challenge` supplies a random one-use nonce bound to user and device. Native `attestKey` hashes `SPVS-ATTEST-v1.<nonce>.<device_id>.<key_id>`. The server decodes bounded CBOR, verifies the pinned certificate chain, validity, basic/key-usage constraints, nonce extension, app-ID hash, environment AAGUID, zero initial counter, credential ID, public-key hash and COSE key consistency. It stores the public key only on success; failed registration consumes its challenge.

6. Each attendance request generates an assertion over SHA256 of the exact signed request bytes. The backend checks the app hash and ECDSA signature and atomically advances a strictly increasing counter under a database lock. Replays and out-of-order assertions cannot pass. Exact authenticated attendance retries return the existing result without double counting.

The verifier is covered by synthetic certificate/signature tests. It has **not** been validated with a genuine Apple attestation object or physical iPhone. Apple receipt risk assessment and distribution-scale fraud monitoring are not implemented. Complete a provider interoperability review before enabling institutional attestation requirements. Apple's root and developer identity are intentionally not fabricated or bundled.

Play Integrity is a fail-closed boundary: no decoded client verdict is trusted. To finish its provider adapter, configure a Google Cloud project linked to Play Console, package identity and allowed signing-certificate digests, obtain a standard Integrity token bound to SHA256 of the exact request, decode it through Google's authenticated server API, enforce request package/hash/timestamp and app/device verdicts, and add replay/quota/error tests on Play-distributed physical devices. This release does not claim that integration is configured or verified.

Presence policy and assurance

Android retains `wifi_unavailable=review|reject|gps_only`. iOS has a separate audited `ios_presence=review|retry|gps_only`, default review. A GPS-only opt-in applies only after mandatory identity, device, liveness and GPS gates; it never overrides failed identity or a required unverified attestation. Every attempt records platform, policy, reported key storage and presence assurance. Missing Wi-Fi is distinct from a mismatched observed network. A permitted iOS current BSSID does not raise GPS to multi-AP fingerprint assurance. Reduced-accuracy location requires retry even when a client reports a small numeric radius. GPS coordinates and Wi-Fi observations remain potentially spoofable client evidence.

Camera protocol and limits

Both camera adapters return processed JPEGs at width 640, front camera with mirror disabled. Android YUV and iOS pixel-buffer formats do not cross the API; Expo handles sensor conversion and orientation before JPEG encoding. ImageManipulator resizes and recompresses, and the server validates JPEG dimensions, lighting, blur, exactly one face, landmarks and 128-dimensional normalized embeddings. The capture loop targets 170 ms intervals, allows a 350 ms prompt transition, and requests slow blink/open-close-open evidence. Actual capture speed and front-camera orientation are hardware dependent. Capture-start timestamps are adjusted by server clock offset; they are signed but not trusted hardware timestamps. Reject gaps outside server bounds and interrupted/expired captures; never forge timing to make a sequence pass.

Server inference and temporal tests do not establish live-camera usability, biometric accuracy, sophisticated presentation-attack resistance or injection resistance. Verify blink capture rate, left/right orientation, lens/mirroring, accessibility, reduced GPS, Wi-Fi throttling, airplane mode, key loss and network retry on real Android and iPhone hardware before a demonstration with students.

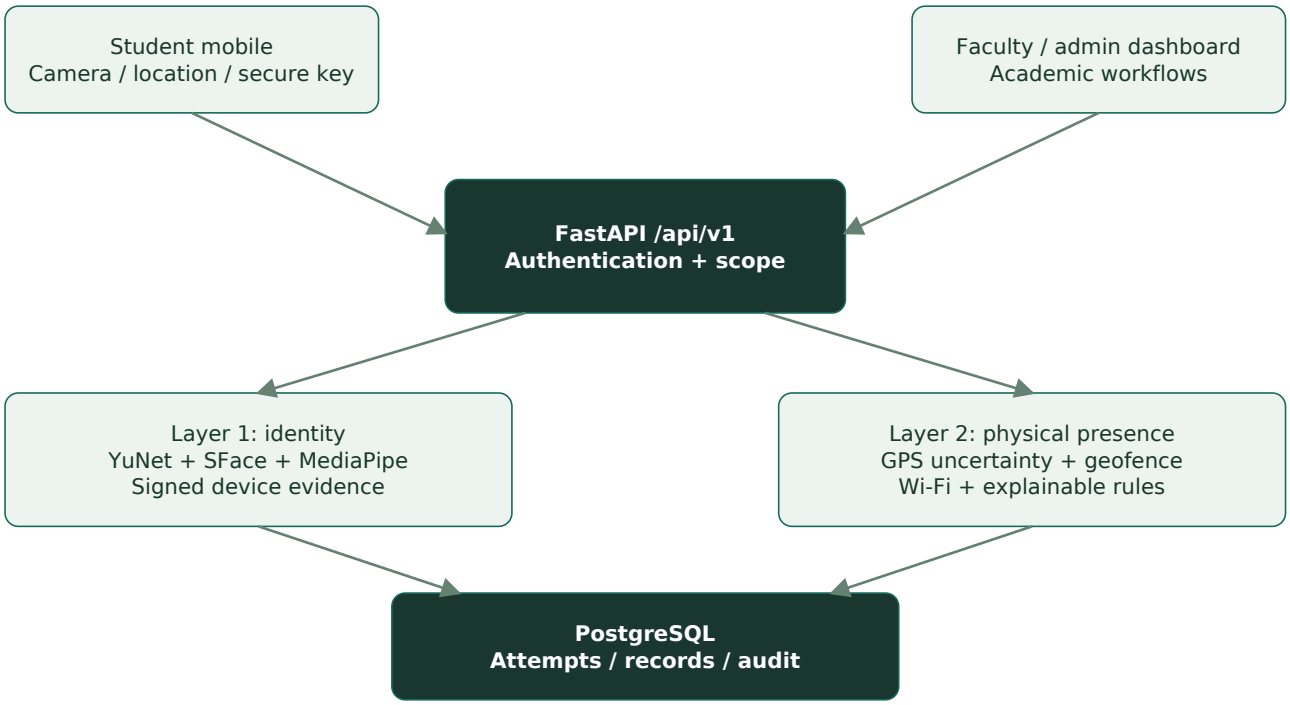
Native build procedures

See SETUP_AND_DEMO_GUIDE.md and `scripts/build_mobile_macos.sh`. Android builds require JDK 17, the SDK/NDK versions emitted by the pinned Expo prebuild and Gradle's trusted repositories. iOS requires macOS, a supported Xcode version (the pinned React Native helper declares at least 16.1; use a current supported Xcode), command-line tools, Ruby/Bundler and CocoaPods. Simulator compilation does not verify Secure Enclave or App Attest. Physical iPhone builds require signing and matching capabilities. TestFlight requires an Apple Developer membership, App Store Connect record, distribution provisioning and a genuine archive/upload. No unsigned file is renamed to IPA.

Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, even if the global attestation requirement is off. The registered platform is client-reported; without this gate an unverified client could select a weaker platform policy. With App Attest unavailable/unconfigured, iPhone verification remains review/retry. GPS-only evidence still has lower presence assurance after app verification.

System architecture

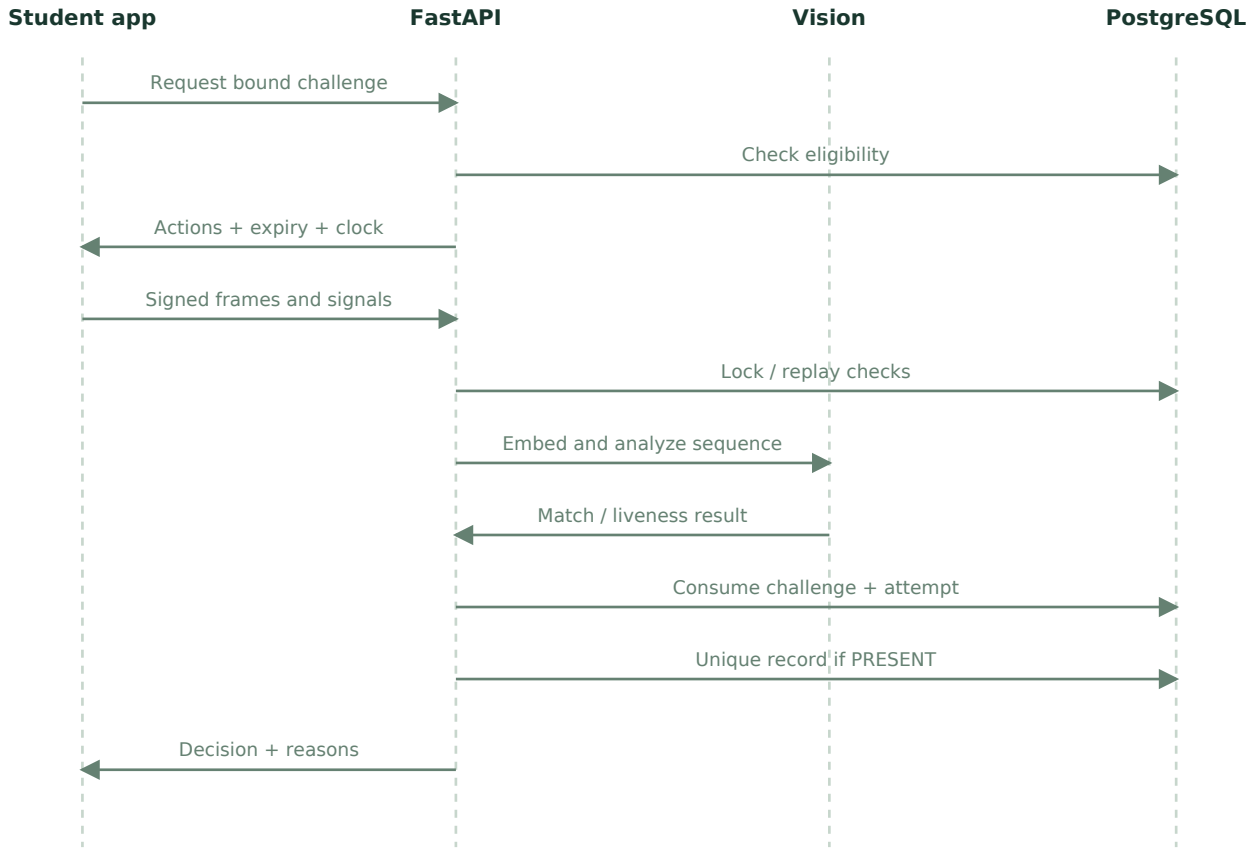
Component architecture



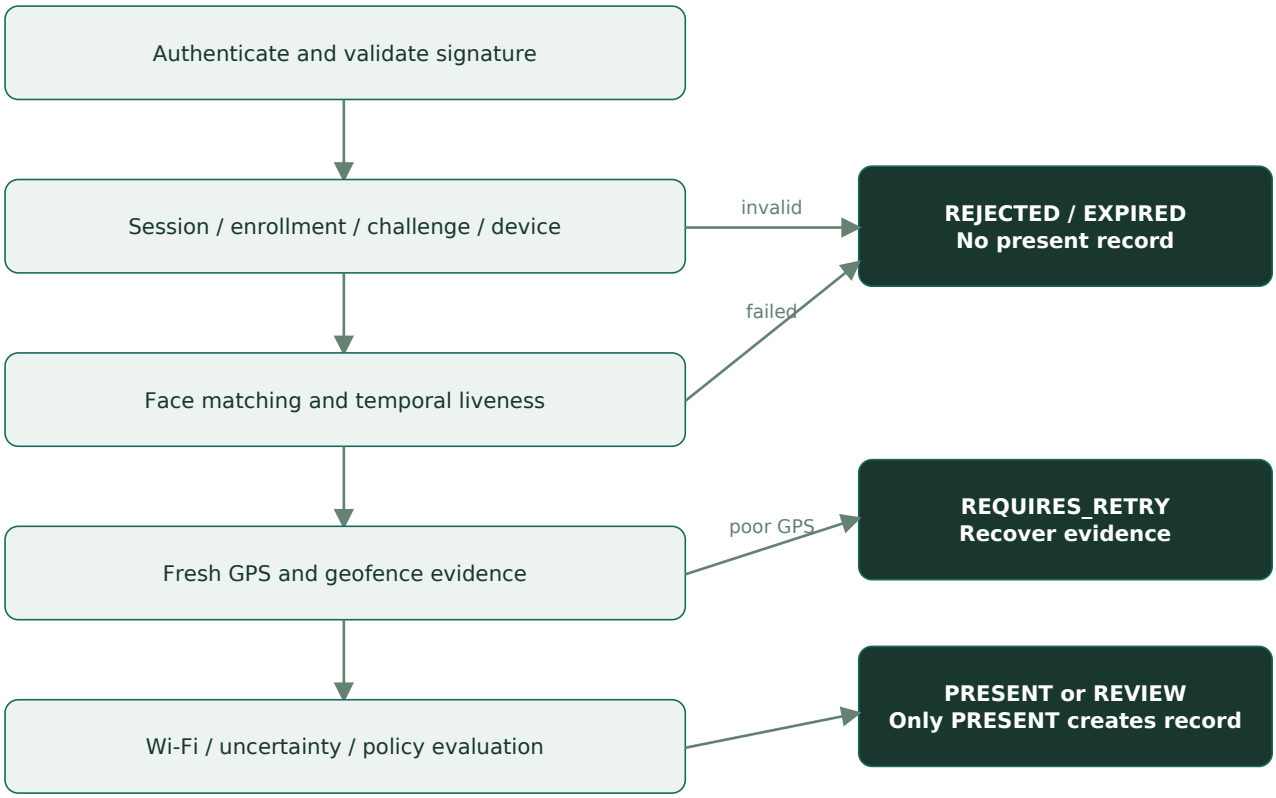
The backend uses synchronous SQLAlchemy sessions in FastAPI synchronous route handlers, which execute in the framework's thread pool. This is deliberate: OpenCV/MediaPipe inference is synchronous and guarded by a process-local model lock. The application does not block an async endpoint with a synchronous database call. Multiple API workers share PostgreSQL rate limits and locks; each worker has its own model memory. Capacity and fair scheduling require load testing before institutional deployment.

The dashboard uses a same-origin `/api` route through Vite in development or nginx in containers. Tokens remain in dashboard memory. The mobile app stores tokens and per-user key metadata in SecureStore; new private keys stay in native Keystore/Secure Enclave storage. Its camera cache files are deleted after conversion; no raw frames are saved by the backend.

Verification sequence



Decision flow



Authentication, malformed payload and signature failures are HTTP errors; signed, attributable verification outcomes are persisted as attempts. This distinction prevents unauthenticated requests from inventing someone else's attempt history.

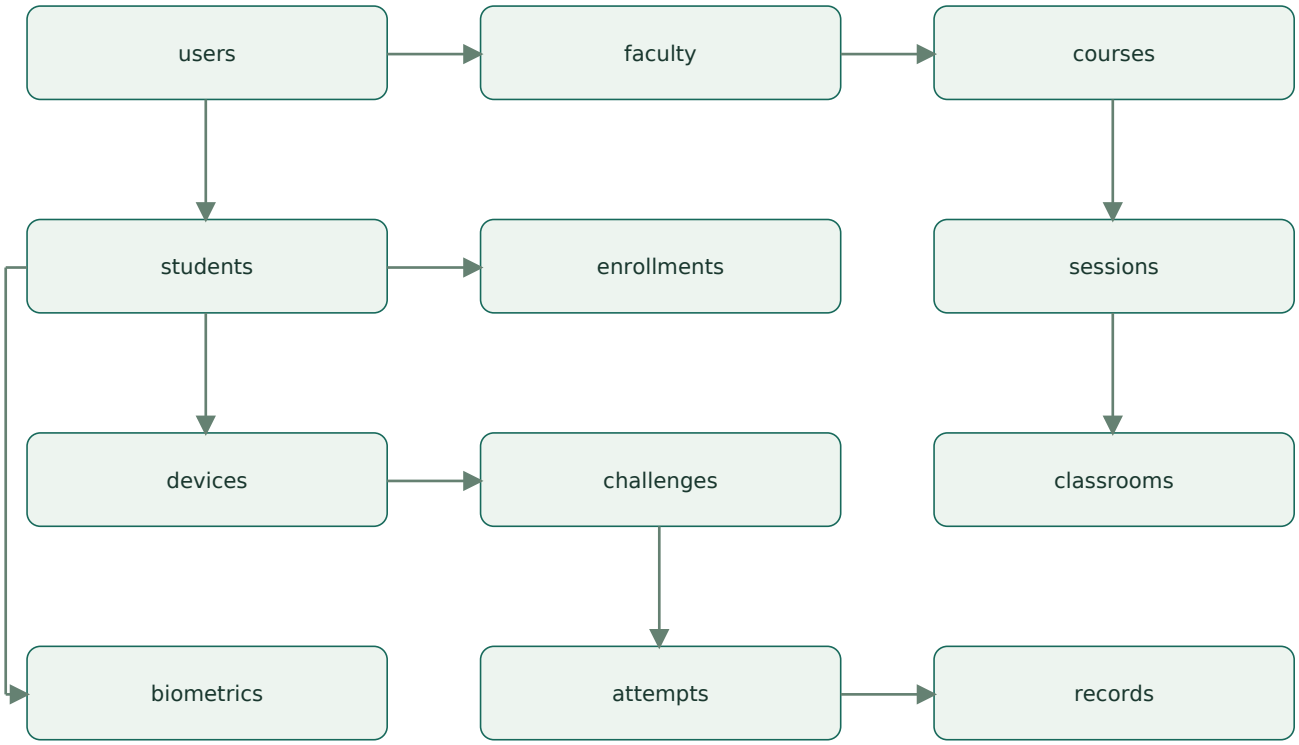
Cross-platform completion update

Native device keys now live in `spvs-security` (Kotlin/Swift), with shared TypeScript adapters. App Attest registration uses dedicated one-use security challenges; assertions bind the exact signed request and advance a PostgreSQL counter. iOS current-network evidence is separate from Android scan measurements. See CROSS_PLATFORM_IMPLEMENTATION.md for trust boundaries and native-build limits.

Database design

PostgreSQL 16 is used in Compose. The initial migration creates 20 application tables plus Alembic's version table. IDs are application-generated UUID strings. Timestamps are timezone-aware. There are no raw-image columns.

Table	Purpose / important constraint
users	Unique email, password hash, current role and active status
students	User FK and unique roll number
faculty	User FK and department
courses	Unique code, assigned faculty FK
enrollments	Composite primary key: course and student
classrooms	Unique name, latitude, longitude, radius
wifi_references	Unique classroom/BSSID and minimum RSSI
policies	Singleton verification settings
devices	Encrypted signing key; partial unique index allows one active device per student
device_resets	Requested reason, status and administrator reviewer
biometric_profiles	Student primary key, encrypted template, model version, consent timestamp
attendance_sessions	Course/classroom FKs, start/end and explicit closure
challenges	Student/device/session bindings, purpose, random actions, expiry and used flag
attendance_attempts	Unique student/idempotency key, payload hash, decision, reasons and evidence summary
attendance_records	Unique student/session, optional attempt FK, status and source
security_events	Actor/session, severity, structured details and review flag
corrections	Record/actor FKs, previous/new status and mandatory reason
disputes	Student/attempt, reason, resolution and reviewer
refresh_tokens	Hashed random token, family, expiry, used and revoked flags
rate_buckets	Shared fixed-window counters keyed by account or connection source



Core foreign-key relationships; full schema and remaining tables listed in the database chapter.

Verification locks the student row, then challenge and session, before consuming the challenge and writing the attempt/record. The same session row serializes a manual correction against verification. Device reset approval locks the student before revocation. A refresh-token rotation locks its token row; reuse revokes the entire refresh family. These transactions complement, rather than replace, unique constraints.

Subtype rows are retained when a current role changes because historical foreign keys must remain valid. Authorization always reads the current user role. Deleting referenced academic entities is intentionally restricted; deactivation and enrollment removal are supported while history remains attributable.

Location summaries retain coordinates rounded to three decimal places for coarse travel anomaly checks, plus distance, accuracy and age. Define an institutional retention schedule before deployment. The current application does not implement automated retention purges or an immutable external audit sink.

Cross-platform completion update

Migrations `5c9d2a1e8b70` and `7a2d13ce4f90` add device key_algorithm/public_key/key_storage, policy ios_presence, security_challenges (user/device/purpose/nonce/expiry/used), app_attest_keys (globally unique key ID, unique device, public key, monotonic bigint counter/environment), and challenge enrollment_request_hash. Existing device rows default to legacy HMAC. The enrollment hash permits exact authenticated HTTP retries; different requests cannot replay a used challenge. Device and student locks serialize revocation/signature decisions. Raw camera frames are never stored in these tables.

API documentation

The authoritative machine-readable contract is `docs/openapi.json`, exported from the implemented application, and the live `/openapi.json`. The versioned API prefix is `/api/v1`. Protected operations use `Authorization: Bearer <access token>`.

Endpoint groups

Group	Main routes	Authorization
Authentication	POST `/auth/login`, `/auth/refresh`, `/auth/logout`; GET `/auth/me`	Public token exchange; authenticated profile
Registration	POST `/auth/register`	Disabled by default; student-only when enabled
Accounts	GET/POST `/users`; PATCH `/users/{id}`	Administrator
Academics	GET/POST `/courses`; GET/POST/DELETE enrollment routes	Course-scoped reads; administrator writes
Classrooms	GET/POST `/classrooms`; PUT `/classrooms/{id}`	Staff read; administrator write
Sessions	GET/POST `/sessions`; GET `/{id}`; POST `/{id}/close`	Enrolled student read; assigned faculty/admin manage
Biometrics	GET/DELETE `/biometrics`; POST `/biometrics/enroll`	Student's own template
Device	GET/POST `/devices`; reset request/list/approval routes	Student ownership; admin reset approval
Evidence	POST `/challenges`, `/attendance/verify`	Authenticated student and active bound device
Review	GET/POST `/disputes`; POST `/{id}/resolve`; GET/POST `/corrections`	Student ownership; scoped staff decisions
Audit	GET `/events`; POST `/{id}/review`	Scoped faculty or administrator
Records	GET `/attempts`, `/records`	Student own data or scoped staff data
Reporting	GET `/overview`, `/reports/attendance.csv`	Scoped staff
Policy	GET/PUT `/policy`	Authenticated read; administrator write with reason

Signing protocol

`POST /attendance/verify` and `/biometrics/enroll` accept the three required fields below and optional App Attest assertion/key fields:

```
{
  "payload": "<exact serialized Evidence JSON string>",
  "signature": "<base64 DER ECDSA/SHA-256 signature>",
  "idempotency_key": "<16 to 64 characters: letters, digits, hyphens or underscores>"
}
```

P-256 ECDSA/SHA-256 signs the bytes of `idempotency_key + "." + payload`; the server verifies with the registered public key. Legacy devices retain the original hexadecimal HMAC protocol during migration. The inner `Evidence` schema has challenge/device IDs, 8–40 timestamped JPEG frames, consent, optional location, real Wi-Fi observations/status and optional integrity token. Coordinates, accuracy and timestamp are measurements; no `distance`, `inside_geofence`, `similarity` or `liveness_passed` field is accepted as proof. Clients should keep the same exact signed request and key after a network timeout to recover the original attendance decision.

Enrollment and attendance purposes are not interchangeable. An enrollment challenge cannot reference a session; an attendance challenge must bind an active, enrolled session and an existing biometric profile. An exact authenticated enrollment retry returns the recorded success while the profile exists; a different request cannot reuse a consumed enrollment challenge. The client can also query `/biometrics` for current status.

Outcomes and errors

An attributable attendance attempt contains `id`, `state`, `reasons`, `risk_score` and `evidence_summary`. `PRESENT` creates one official record. `PENDING_REVIEW`, `REQUIRES_RETRY`, `REJECTED` and `EXPIRED` do not create a present record. Faculty may create an explicitly sourced correction later.

HTTP errors use `{"error":{"code":403,"message":"..."}}`. Validation responses include field locations without echoing sensitive input. Typical codes are 401 authentication failure, 403 role/device/signature denial, 404 missing resource, 409 conflict/replay, 413 body limit, 422 invalid evidence, 429 rate limit and 503 missing verification service/models. Never treat an HTTP failure as an attendance approval.

Most operational lists expose the latest 500 rows; course and enrollment lists are not paginated. CSV export includes all records in the authorized selection. Large institutions need cursor pagination, query optimization and retention jobs before deployment.

Cross-platform completion update

`POST /devices/registration-challenge` issues a one-use nonce. `POST /devices` accepts public_key (base64 canonical P-256 SPKI), registration_challenge_id, registration_signature and reported key_storage in addition to password/label/platform. New registrations do not return a shared secret. See CROSS_PLATFORM_IMPLEMENTATION.md for exact signing bytes. `GET /attestation/capabilities`, `POST /attestation/challenge`, and `POST /attestation/register` implement the App Attest boundary. Signed requests may contain outer attestation_key_id and attestation_assertion; the assertion covers the exact idempotency/payload bytes. Evidence adds accuracy_authorization, wifi_source and current_bssid. `PUT /policy` accepts ios_presence=review|retry|gps_only with an audit reason. `GET /admin/devices` is administrator-only and explicitly labels reported storage versus verified app registration. The regenerated openapi.json is the field-level contract.

Biometric design

Model provenance and separation of responsibilities

YuNet detects a face and five alignment landmarks. It does not establish identity. SFace generates a 128-dimensional learned face representation, using OpenCV's `FaceRecognizerSF.alignCrop` and `feature` APIs. MediaPipe Face Mesh separately supplies dense facial landmarks for temporal liveness. ONNX models are executed by OpenCV DNN on CPU; ONNX Runtime is unnecessary for this pipeline.

Asset	Provenance	License
`face_detection_yunet_2023mar.onnx`	OpenCV Zoo, commit `48b10fa505aeb51fbd127fc60a5c660040e823b7`	MIT; Shiqi Yu notice retained
`face_recognition_sface_2021dec.onnx`	Same official repository/commit	Apache 2.0; full license retained
`face_landmarker.task` float16 v1	Official Google MediaPipe model storage; consumed by MediaPipe 1.1.1 Tasks	MediaPipe code is Apache 2.0; consult the linked official model card/provider terms for the pretrained asset

Expected SHA-256 values are in `scripts/download_models.py`, copied from the official Git LFS pointer objects. A mismatch stops installation. The SFace asset is 38,696,353 bytes and YuNet is 232,589 bytes. No model is trained in this project.

The landmark task is downloaded over verified HTTPS from the versioned official Google URL. Its SHA-256 is pinned as `64184e229b263107bc2b804c6625db1341ff2bb731874b0bcc2fe6544e0bc9ff`; it is loaded in IMAGE mode with at most two faces. The current MediaPipe Tasks API avoids the legacy package's protobuf dependency.

Enrollment and comparison

Enrollment requires an authenticated student, an authorized device, explicit consent, a fresh enrollment-purpose challenge and a server-validated sequence. JSON cannot carry a trusted embedding or similarity score. JPEG input is base64-decoded, limited to dimensions 160–1280, checked for brightness and Laplacian sharpness, and must contain exactly one usable YuNet face of at least 70 pixels. The aligned BGR crop passes through OpenCV's SFace preprocessing. Returned vectors must contain exactly 128 finite values and have nonzero norm. They are L2-normalized.

Every verification frame is embedded and compared against the encrypted enrolled template. Enrollment frames must match the first frame so that a sequence cannot trivially switch people. The minimum similarity across frames must meet `FACE_THRESHOLD`, default **0.5**. OpenCV's tutorial gives a cosine baseline of approximately 0.363 for its example evaluation; 0.5 here is a stricter initial operating point, not a locally calibrated false-acceptance guarantee. A realistic institution must collect consented evaluation data and measure FAR/FRR across lighting, pose and user groups before choosing a threshold.

The enrollment template is the normalized mean of sequence embeddings and is encrypted with Fernet under a separate `BIOMETRIC_KEY`. Templates carry an explicit model version. Raw frame bytes and unencrypted embeddings are not persisted. Student deletion requires password confirmation. Backup retention can still retain deleted ciphertext; operators must apply their retention policy to backups as well.

Liveness protocol

The server issues an unpredictable UUID challenge bound to a student, device, purpose and attendance session (when applicable). It selects a left/right turn and randomizes the turn/blink ordering, inserting center poses. A challenge normally expires after 60 seconds and is consumed once. Session attempts and challenge issuance are rate limited.

Face Mesh computes eye aspect ratio (EAR) from six landmarks per eye, averaged across both eyes. A blink must transition open (`EAR > 0.23`) → closed (`EAR < 0.19`) → open. Head direction is a signed nose displacement relative to eye centers, normalized by interocular distance. Center is absolute displacement below 0.12; turns exceed 0.18 in the requested direction. Two consecutive observations must support each non-blink pose. This is a heuristic displacement proxy rather than a calibrated three-dimensional head-pose estimator.

A sequence contains 8–40 frames, has increasing timestamps separated by 0.08–2.5 seconds, spans at least two seconds, remains inside the challenge time window, and may not claim a final capture more than two seconds in the future. Identical encoded frames are rejected. Mobile captures unmirrored camera images and synchronizes capture timestamps to the challenge's server clock. Direction conventions, timing and EAR thresholds require physical-camera validation; basic frame rates can miss a fast blink, so the UI asks for a slow close/open action.

Limits and measured evidence

Landmark liveness rejects a static frame and some naive replay attempts. It does not prove camera provenance and can be defeated by sophisticated replay, virtual-camera injection, deepfakes or a compromised client. Randomized action order and signatures increase the effort but are not certified presentation-attack detection. Client timestamps are assertions checked for consistency, not trusted hardware timestamps.

Real model tests use the public NASA astronaut image from scikit-image and Grace Hopper image from matplotlib: same-face photometric variation, different-person comparison, zero faces and multiple faces. These smoke tests validate model operation; four tests on two portraits are not a biometric accuracy study. No live student was enrolled during automated evaluation.

Cross-platform completion update

Both platforms submit the same processed unmirrored JPEG format and randomized action order. The client cancels backgrounded capture, checks challenge expiry before/after each frame, deletes temporary images and translates quality/no-face/sequence errors into retry guidance. The server rejects non-finite landmark geometry and tests dark/overexposed frames using the real inference pipeline. Do not infer physical-camera frame rate or left/right orientation from JS export success. Calibration must use separate consented development and held-out evaluation participants; stratify lighting, eyewear, skin tones, device/OS and accessibility needs. Measure false match/non-match rates with confidence intervals and failure-to-acquire, not a single invented accuracy figure. Keep raw consented evaluation data outside Git with access controls, deletion dates and a non-biometric review route; freeze thresholds only after review.

Security architecture

Authentication and authorization

Passwords use Argon2id with memory cost 65,536 KiB, time cost 3 and parallelism 2. Failed logins use a constant-work password check against a dummy hash when the account is absent. Tokens are HS256 JWTs with explicit issuer, audience, type, issued-at, expiry, subject and random ID; the secret must be at least 32 characters and is randomly generated by setup. Access lifetime defaults to 15 minutes. Refresh tokens are random 48-byte URL-safe values stored only as SHA-256 hashes, with a default seven-day lifetime. Rotation is transactional; reused tokens revoke their refresh family.

Logout revokes refresh sessions. Already-issued access tokens remain valid until expiry unless the account is disabled; endpoints check active status and current role on every request. This access-token revocation window is documented, not concealed. Role updates revoke refresh tokens and device bindings and are administrator-only with reasons. The dashboard keeps credentials in memory, so reload requires login. Mobile credentials use OS secure storage.

Device binding

New registration verifies a password and one-use nonce proof from a native P-256 key. The private key remains inside Android Keystore or iOS Secure Enclave (Keychain for simulator); only canonical SPKI public-key bytes reach the server. Evidence signs exact UTF-8 `idempotency_key + "." + payload` with ECDSA/SHA-256 and a base64 DER signature. Existing HMAC devices remain compatible; new legacy registrations are disabled by default.

The database has a partial unique active-device index. Approved reset revokes old devices; reinstall/key loss cannot silently create a replacement authorized key. Reported hardware backing is not independent key attestation. App Attest verifies a configured pinned chain, app ID, nonce, environment and public key, then checks assertions with a strictly increasing counter. Genuine Apple interoperability and physical-device evaluation remain outstanding. Play Integrity remains unconfigured and fails closed. Never forward a client boolean as a verdict. Full setup and verification limits are in CROSS_PLATFORM_IMPLEMENTATION.md.

Evidence and decisions

Identity gates precede presence scoring. Enrollment, session, face, challenge and device failures cannot be overridden by a good GPS score. Server-owned rules calculate distance and signal match. A fully outside uncertainty circle rejects; an overlapping boundary requests review. Missing/inaccurate/stale data requests retry or follows the explicit Wi-Fi policy. Mock-location flags reject when reported, but the absence of a flag does not prove an honest OS.

The verified request atomically consumes its challenge, stores an attempt and optionally inserts one attendance record. Retries with the same user/key and exact payload return the original attempt; modified payloads conflict. A different key cannot reuse a consumed challenge. Database locks serialize concurrent retries. Manual corrections are distinct records with actor, reason and before/after status.

Storage and operations

Fernet protects templates and signing credentials at rest with a dedicated key. Secrets, caches and generated local state are excluded from archives. There is no raw-image retention endpoint. Location summaries are coarse and restricted by role. Production operators should use separate least-privilege migration/runtime database accounts, secret management, key rotation with re-encryption, a retention process, encrypted backups and trusted TLS termination.

Rate buckets live in PostgreSQL, shared across workers. The API does not trust caller-supplied forwarded headers for rate identity; behind a reverse proxy, connection IP limits can group multiple clients. Account limits remain independent. Configure trusted proxy handling and infrastructure limits for deployment. Bounded request bodies, decoded image dimensions and model serialization reduce resource abuse but do not replace load testing or upstream request throttling.

Audit rows are append-only through ordinary application APIs except their reviewed flag. A database administrator can still alter them: this is not tamper-proof external audit storage. Implement a protected external append-only sink if that is an institutional requirement.

Cross-platform completion update

New clients use native non-exported P-256 keys and proof of possession. Android hardware backing is queried locally; physical iPhones require Secure Enclave and simulators use device-only Keychain. Storage metadata alone is not server-verified hardware attestation. App Attest verification enforces a configured pinned Apple root, app identity, nonce, environment, key binding and increasing assertion counter. Synthetic certificates verify logic only; actual Apple-provider acceptance remains unverified. Android Play Integrity remains fail closed. Legacy HMAC registration is disabled by default, existing keys migrate through audited resets. Exact enrollment retries recover a lost HTTP response without permitting different replay payloads. Required attestation cannot be replaced by a favorable GPS score.

Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, even if the global attestation requirement is off. The registered platform is client-reported; without this gate an unverified client could select a weaker platform policy. With App Attest unavailable/unconfigured, iPhone verification remains review/retry. GPS-only evidence still has lower presence assurance after app verification.

Threat model

Assets include student identity, biometric templates, device credentials, location evidence, attendance integrity, refresh sessions and faculty audit accountability. Trust boundaries are the mobile device to HTTPS API, staff browser to API, API to PostgreSQL, and operator secrets to server runtime. A student client is untrusted even after login.

Threat	Mitigation in this implementation	Residual risk / evaluation
Shared QR/token	No standalone shared token can create attendance; enrolled identity, challenge and device are required	Account and device credential collusion remains possible
Face photo	Actual embedding plus randomized temporal eye/head sequence	Advanced replay/deepfake/injection can mimic landmarks
Client says liveness passed	Schema rejects unknown verdict fields; server analyzes frames	Frame capture provenance is not attested
Hostel or remote attempt	Server GPS geofence, accuracy, age and available Wi-Fi	Compromised phones may forge all signals
GPS spoofing	Mock indicator, stale/accuracy checks, coarse travel anomaly	An omitted/forged mock flag is not trustworthy
Device ID cloning	Native P-256 proof and signed evidence; one active binding; legacy HMAC compatibility	Legacy secrets can be copied; native storage reports are not independent key attestation
Replay or duplicate	Purpose/user/device/session binding, expiry, single use, unique keys and records	Encoded video may be re-rendered to evade byte equality; liveness is basic
Concurrent submission	Student/challenge/session locks and unique constraints	Session-level serialization limits throughput
Faculty accesses another class	Server course-ownership checks and query scoping	Privileged administrator has institution-wide access
Silent attendance edit	Mandatory reasons, correction ledger, structured events	Database-superuser tampering is outside app controls
Refresh-token theft/reuse	Secure storage, hashing, rotation, family revocation	Access tokens have an expiry window; browser XSS can act while logged in
CSV formula injection	Escape dangerous leading formula characters	Downstream spreadsheet handling still needs care
Large payload/decoded image	Actual-byte body bound, JPEG byte and dimensions cap	Valid expensive requests still consume inference capacity
Dependency compromise	Lockfiles, verified model hashes, audit reports	Audits are time-bound and cannot prove absence of defects

Abuse evaluation protocol

Use consented volunteers on both real Android and iPhone devices. Test identical-photo presentation, screen replay, a wrong person, multiple faces, same evidence/key replay, same evidence/new key replay, a second registered device, a revoked device, a distant location, disabled location permission and unavailable Wi-Fi. Inspect both HTTP outcomes and resulting database/audit records. Record model version, threshold, device model, OS version, illumination and capture timing. Never label a human review or an unrun test as successful automatic presence verification.

The public test portraits are used only for model smoke tests. No external biometric service receives student images, and this implementation does not use facial recognition for surveillance or sensitive-trait inference.

Cross-platform completion update

New threat cases: App Attest development/production confusion, attacker-selected trust roots, certificate nonce substitution, assertion-counter replay, app reinstall with persistent iOS Keychain entries, and presenting iOS BSSID identity as multi-AP RSSI evidence. Controls include server-configured root fingerprint/app ID/environment, challenge consumption and key uniqueness, atomic counters, installation-ID checks, and separate server iOS policy. Synthetic adversarial tests cover protocol rejection; physical sensor injection, rooted-device behavior and sophisticated presentation attacks still require evaluation. Keychain-only simulator keys are never claimed to be Secure Enclave keys.

Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, even if the global attestation requirement is off. The registered platform is client-reported; without this gate an unverified client could select a weaker platform policy. With App Attest unavailable/unconfigured, iPhone verification remains review/retry. GPS-only evidence still has lower presence assurance after app verification.

Final test and build results

Suite	Passed	Failures/errors	Skipped
Backend	109	0	0
Mobile	28	0	0
Dashboard	8	0	0
Browser	3	0	0

Total: **148 passed**. See FINAL_COMPLETION_REPORT.md for test boundaries, commands and intermediate failures. Baseline evidence is retained separately.

Capability	Observed status
Android debug APK	PASS: genuine Gradle assembleDebug, arm64-v8a; debug signed, development client; physical-device testing NOT RUN
Android/iOS JS export	PASS: separate platform exports; final combined export in evidence
iOS native project	PASS: Expo prebuild, camera/location/privacy settings, local podspec/autolinking and static configuration checks
iOS native compilation / simulator / IPA	NOT RUN: Linux host; no macOS/Xcode or Apple signing identity
PostgreSQL migrations	PASS: fresh upgrade, check, full downgrade, re-upgrade, check on isolated PostgreSQL 16 database
Backend and dashboard containers	PASS: both images built and used for real browser/API tests
Physical Android / physical iPhone	NOT TESTED
Apple App Attest	Synthetic verifier/counter tests passed; genuine provider/device verification UNVERIFIED
Google Play Integrity	NOT CONFIGURED; fail-closed boundary only
Biometric calibration	NOT PERFORMED; six real-model smoke cases are not accuracy metrics

No physical Android/iPhone, iOS simulator, Xcode or genuine provider-attestation result is claimed. Synthetic API/sensor/certificate inputs and mocked React Native components are identified in their test modules.

Requirements traceability

Requirement	Implementation	Automated evidence	Remaining validation
Persistent normalized storage	20 SQLAlchemy tables + explicit Alembic migration	Migration/table/constraint tests on PostgreSQL	Deployment backups and restore drill
Password/JWT/refresh security	`core/security.py`, `api/auth.py`	Login, invalid JWT, rotation, reuse and logout tests	Operational key rotation
Backend role restrictions	`roles`, `access_course`, `session_ids`	Student/staff denial, cross-course checks and role-change tests	Institution-specific account lifecycle
Academic management	`api/academic.py`, dashboard courses/classrooms/sessions	CRUD/session lifecycle and browser workflow	Real campus data entry
Actual face recognition	YuNet + SFace in `services/vision.py`	Real same-face/different-person/no-face/multi-face tests	Consented local calibration
Enrollment consent/deletion	`/biometrics/enroll`, DELETE `/biometrics`	API consent/storage/deletion contract tests	Live student enrollment
Temporal liveness	MediaPipe landmarks + sequence validator	EAR/pose sequence, timing, expiry tests	Physical phone spoofing/usability matrix
Device binding	Native P-256 keys + nonce proof + unique active-device index	Signature/device mismatch, second-device denial and reset tests	Physical Keystore/Secure Enclave evaluation
Platform integrity	App Attest native/server verifier; Play Integrity fail-closed boundary	Unconfigured policy never passes as verified	Genuine provider accounts, physical-device interoperability and receipt risk evaluation
GPS evidence	Haversine + uncertainty/freshness gates	Boundary/stale/inaccurate/mock/travel tests	Indoor GPS field measurements
Wi-Fi evidence	Native Android scan module + reference matching	Matching/weak/unavailable policy tests	Kotlin build and real AP scans; see current report
Deterministic decision	`/attendance/verify` + presence service	Hard-gate and decision API tests	Full real-device happy path
Concurrency/idempotency	Transaction locks + unique constraints	Simultaneous identical requests, replay and duplicates	Load testing
Auditing/anomalies	Structured events and correction ledger	Repeated failures, device reset, policy/role/correction audit	External immutable sink if required
Faculty reports	Aggregates, Recharts and scoped CSV	CSV escaping, live browser export, actual API counts	Large data export performance
Student disputes	Mobile history + scoped dispute API	Ownership and resolution tests	Real student support process

Requirement	Implementation	Automated evidence	Remaining validation
Dashboard quality	React, router, query, labeled forms and responsive CSS	Interaction tests, production build, Chromium workflow	Assistive technology and usability study
Student app integration	Expo camera/location/Secure Store/API signing	TypeScript, HMAC/boundary tests, Metro/Hermes export	Physical sensor/permission tests
Documentation/artifacts	Markdown, diagrams, PDF/ZIP packaging script	PDF text/page and ZIP CRC/contents checks	Fill academic identity placeholders

“Automated” in this table does not imply manually tested. The generated verification report identifies the current counts and commands. No fake biometric approval or sensor measurement is used by application code.

Cross-platform completion update

Added requirement	Implementation	Verification boundary
Android/iOS native device keys	spvs-security, device_keys.py, registration nonce	Real crypto/API tests; physical key storage untested
iOS presence policy	ios_presence, current_bssid, accuracy_authorization	PostgreSQL/API/unit and admin browser tests
App Attest	Swift DeviceCheck bridge, app_attest.py, key/counter tables	Synthetic chain/assertions; genuine provider setup outstanding
Cross-platform camera state machine	capture.ts, shared UI and error guidance	Logic/component tests; real capture untested
Reproducible native projects	config plugin, podspecs, macOS build script	Prebuild/config/autolinking; native outcomes in final report
Enrollment recovery	enrollment_request_hash	Exact retry and different-key replay tests

Known limitations and release boundary

Verification assurance

- A normal phone can be rooted, instrumented or used with injected camera/location data. A native device signature (or a legacy HMAC) proves key possession, not trustworthy physical capture.

- Temporal landmark liveness is a heuristic. It is not ISO-certified presentation-attack detection, and sophisticated replay or deepfake attacks may succeed. The action vocabulary is small even though challenge IDs and ordering are randomized.

- SFace is a real pretrained model, but the chosen threshold has not been calibrated on this institution. The public-portrait smoke tests do not estimate FAR, FRR, fairness or operational accuracy.

- Real-camera head-direction conventions, blink timing, enrollment usability and sustained phone-to-server attendance must be validated on physical devices. Automated transaction tests use an explicitly substituted vision component; they do not replace that evaluation.

- GPS and Wi-Fi support approximate physical presence, not absolute classroom proof. Rounded location history supports only coarse anomaly rules. The absence of a mock-location flag does not establish honesty.

- Wi-Fi scans are implemented only through a local Android module and depend on OS permissions, enabled location, recent scan availability and throttling. iOS may provide a permitted current BSSID only; no RSSI scan is fabricated. The separate default iPhone policy requires review.

- Play Integrity remains an unconfigured provider boundary. App Attest has native and server code, but Apple configuration and genuine provider/device validation remain outstanding. Requiring unverified attestation moves otherwise eligible attempts to review.

Application and operational limits

- Web lists generally show the latest 500 entries. Large datasets need cursor pagination and indexed query tuning; exports can be large.

- Session-row locks serialize verification and manual correction within a session. This prioritizes consistency and requires throughput evaluation before large lectures.

- Inference is CPU-based and serialized per worker. Model memory is repeated across workers; no measured capacity or latency SLA is claimed.

- Dashboard access tokens live in memory. Reload requires login. Logout revokes refresh tokens; existing access tokens have a maximum 15-minute default expiry window unless the user is disabled.

- Exact signed enrollment retries recover a successful response; different replay requests conflict. Attendance retains durable idempotency.

- Audit records are protected through application authorization, not an external tamper-proof log. A database administrator can modify storage.

- No automated retention purge, key rotation/re-encryption job, institution SSO, password reset/email service or external audit export sink is included. Identity/role provisioning is institution-managed in the dashboard.

- In a real college, biometric consent, an accessible alternative attendance process, retention and dispute policies require institutional decisions. This software does not supply those policies.

- Demo Compose uses a database-owner account, local development secrets and no TLS reverse proxy. Harden deployment according to `DEPLOYMENT_GUIDE.md` before using real student data.

Dependency audit boundary

The current backend and dashboard audit results are recorded in `TEST_RESULTS.md` and the verification report. The Expo dependency tree currently contains two upstream advisories without published fixes: `braces` nested-pattern stack exhaustion (GHSA-vfj7-8cjw-p6xm) and `node-forge` RSA signature verification (GHSA-86w9-cpqp-85rv). npm propagates these through 15 affected package entries; that is not 15 distinct root flaws. These concern Metro/build tooling and Expo certificate utilities in this project. The application's HMAC implementation uses `@noble/hashes` and the server uses Python cryptography. Do not expose the development bundler to untrusted networks or treat this explanation as a general security clearance. Re-audit before distribution and adopt upstream fixes when available.

See the verification report for exact build, native compilation and physical-device test status. An exported JavaScript bundle is not an installed/tested Android APK, and Linux validation is not macOS ARM64 validation.

Historical native build blocker

The baseline release failed downloading the React Native Android AAR with HTTP 403. This completion explicitly configures the managed proxy and CA trust, retaining TLS verification. The latest actual build outcome supersedes that baseline and is recorded in FINAL_COMPLETION_REPORT.md. JavaScript exports are reported separately from native compilation and physical-device testing.

Cross-platform completion update

The current native build outcomes are in FINAL_COMPLETION_REPORT.md. No physical Android or iPhone run, Xcode compilation, simulator interaction, Apple Silicon execution, genuine Apple attestation registration, Play Integrity provider verification, institutional deployment or population biometric calibration is claimed. App Attest certificate/assertion logic is implemented and synthetic-tested; Apple receipt risk analysis is not implemented. CocoaPods resolution awaits macOS; no validated Podfile.lock is invented. `braces` <=3.0.3 and `node-forge` <=1.4.0 remain affected upstream; official registry latest versions were still 3.0.3 and 1.4.0 during this run. npm reports 15 affected mobile package entries from these two root advisories. Do not use npm force-fix downgrades to incompatible Expo 44/React Native 0.72. Keep Metro/dev tooling private, build only trusted source and do not use its vulnerable certificate utilities as an institutional trust authority. Backend and dashboard audit results are recorded separately.

Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, even if the global attestation requirement is off. The registered platform is client-reported; without this gate an unverified client could select a weaker platform policy. With App Attest unavailable/unconfigured, iPhone verification remains review/retry. GPS-only evidence still has lower presence assurance after app verification.

SPVS final completion and verification report

Completion branch: `spvs/final-completion-cross-platform`, based on `spvs/complete-implementation-20261022` at `b19777df3cb3210330075c702e5e98f487213a5d`. Reporting date: 23 October 2026 (Asia/Kolkata). Source/delivery commits and PR are recorded in RELEASE_CHECKPOINT.md after publication.

Release assessment

A cross-platform implementation and verified development package are delivered. **Institutional production readiness is not established.** Physical-camera attendance on Android/iPhone, Xcode compilation, genuine Apple App Attest, Play Integrity provider integration, consented biometric calibration and unresolved mobile upstream advisories remain release gates. An Android build or synthetic test must not be read as physical-device verification.

Baseline independently reproduced

The starting checkout was clean and matched the supplied commit. The original 72 backend tests (real PostgreSQL and four real-model smoke cases), 8 dashboard tests and 7 mobile tests passed. The original browser workflows were rerun within the expanded final three-test browser suite. The prior Android dependency blocker was reproduced, then the managed HTTPS proxy and CA trust were explicitly supplied to the JVM; TLS verification was never disabled and only trusted dependency repositories were used.

Implemented changes

- Native P-256 device keys: Android Keystore, physical-iPhone Secure Enclave and simulator Keychain; exact-byte signatures and nonce-based proof of possession; legacy HMAC compatibility with new legacy registrations disabled by default.

- Additive PostgreSQL migrations for device keys, one-use security challenges, App Attest keys/counters and separate iOS policy. Device signature checks participate in row locking with revocation. Exact enrollment retry recovers a lost HTTP response; different request replay is denied.

- Apple App Attest support detection, key generation, attestation and assertion bridge; backend pinned-root chain/nonce/app/environment/key checks and monotonic counters. Genuine Apple configuration and validation are still required. Google Play Integrity remains unconfigured and never reports success.

- iOS current-network BSSID through the permitted NetworkExtension API, no fabricated RSSI or AP scans; precise/reduced location authorization and fresh fix handling; default iOS review with explicit audited retry/GPS-only choices. Automatic iPhone GPS-only approval additionally requires a verified App Attest assertion, preventing unverified platform claims from selecting a weaker automatic policy.

- Shared camera challenge state machine with ordering, expiry/background cancellation, temporary-image cleanup and actionable biometric errors. Same actual backend models and mandatory identity gates serve both platforms.

- Reproducible Expo configuration, local module podspecs/autolinking, iOS permission strings/entitlements/minimum OS, icons, secure release transport defaults, Android backup exclusion, EAS profiles and Apple Silicon build scripts.

- Dashboard policy controls and actual device capability list distinguish reported hardware storage, legacy signing and verified App Attest registration.

- Expanded protocol, PostgreSQL, model, UI and browser tests; updated architecture/security/API/database/user/developer/academic documentation, calibration method and verified final artifacts.

Final automated test results

Suite	Passed	Failures/errors	Skipped
Backend	109	0	0
Mobile	28	0	0
Dashboard	8	0	0
Browser	3	0	0

148 passed, with failed/skipped counts shown above. JUnit evidence is in `docs/evidence/final`. Backend API happy paths explicitly substitute vision only in tests; real models are independently executed on public reference portraits/no-face/multiple-face/quality inputs. Synthetic certificate chains test verifier logic, not Apple-provider acceptance. React Native host components and sensors are mocked for logic/component tests; no simulator or phone interaction is represented by those results.

Builds and test matrix

Capability	Observed status
Android debug APK	PASS: genuine Gradle assembleDebug, arm64-v8a; debug signed, development client; physical-device testing NOT RUN
Android/iOS JS export	PASS: separate platform exports; final combined export in evidence
iOS native project	PASS: Expo prebuild, camera/location/privacy settings, local podspec/autolinking and static configuration checks
iOS native compilation / simulator / IPA	NOT RUN: Linux host; no macOS/Xcode or Apple signing identity
PostgreSQL migrations	PASS: fresh upgrade, check, full downgrade, re-upgrade, check on isolated PostgreSQL 16 database
Backend and dashboard containers	PASS: both images built and used for real browser/API tests
Physical Android / physical iPhone	NOT TESTED
Apple App Attest	Synthetic verifier/counter tests passed; genuine provider/device verification UNVERIFIED
Google Play Integrity	NOT CONFIGURED; fail-closed boundary only
Biometric calibration	NOT PERFORMED; six real-model smoke cases are not accuracy metrics

Test category	Android	iOS
Shared UI/protocol and platform policies	Automatically tested	Automatically tested
Native-module project generation	Automatically checked	Automatically checked
Native compilation	See actual APK outcome above	Not tested (no Xcode)
Simulator/emulator interaction	Not tested	Not tested
Physical sensors/enrollment/attendance	Not tested	Not tested
Provider-backed attestation	Not tested	Not tested

Security review and dependency findings

Argon2id, scoped JWT authorization, refresh rotation/reuse detection, course scope, duplicate attendance constraints, challenge replay controls, audited correction/reset flows, shared database rate limiting, bounded images/request bodies and encrypted templates were preserved and regression-tested. Native private keys no longer cross the JavaScript/network boundary for new registrations. Reported hardware storage is not independent key attestation. App Attest binds the app instance and signed request; it does not establish truthful GPS/camera sensors or prove a separately generated signing key is hardware-backed.

The current Python runtime-lock audit reports no known vulnerabilities; the dashboard npm audit reports zero. The mobile npm audit reports **15 high-severity affected package entries from two root advisories**: GHSA-vfj7-8cjw-p6xm (`braces` <=3.0.3) and GHSA-86w9-cpqp-85rv (`node-forge` <=1.4.0). Official registry latest versions remained 3.0.3 and 1.4.0. npm's suggested Expo 44 / React Native 0.72 downgrades are incompatible with this application and were not applied. Keep dev tooling private and re-audit for compatible upstream fixes. No audit is a guarantee against unknown flaws or container/OS vulnerabilities.

Apple trust settings are operator configuration, not supplied by a client. Genuine certificate/receipt risk evaluation and provider interoperability remain outstanding. Raw biometric frames are transient and excluded from artifacts; encrypted templates are deletable. Automated retention purge, deployment key rotation, externally immutable audit storage, load testing and institutional privacy/accessible-alternative decisions remain operational work.

Commands and observed corrections

- `bash scripts/backend.sh pytest --junitxml=../work/final/backend-tests.xml` against isolated PostgreSQL; model assets loaded from verified existing files.

- `npm test -- --run` and `npm run build` in dashboard; `npm run typecheck` and expanded Vitest/component tests in mobile.

- `npx playwright test` in dashboard against actual backend/dashboard/PostgreSQL containers; admin session/CSV, faculty scope and iPhone policy/audit workflows.

- `npx expo prebuild --no-install --platform all`; `python3 scripts/verify_native_config.py`; Expo autolinking resolve for Apple found both local podspecs.

- `npx expo export --platform android`, `--platform ios`, and final combined export; no JS export is counted as native compilation.

- `./gradlew :app:assembleDebug --no-daemon --max-workers=2 -PreactNativeArchitectures=arm64-v8a` with managed proxy and CA trust.

- `docker build` for backend/dashboard with a CA secret; real `/health` and `/ready` endpoint checks.

- Fresh database `alembic upgrade head`, `check`, `downgrade base`, `upgrade head`, `check`; no schema drift.

- `npm audit --json` for both clients and `pip-audit --disable-pip --no-deps` against the frozen runtime requirements.

- `expo install --check` online was blocked by the cloud proxy; offline bundled-manifest comparison reported dependencies up to date, explicitly weaker than an online check.

- A resumed workspace had stopped PostgreSQL; its resulting test-fixture errors were resolved by restarting the container and rerunning tests. A browser assertion initially used POLICY_CHANGED rather than the UI's POLICY CHANGED; the corrected suite passed. Neither intermediate failure is concealed as a passing run.

Artifact verification and reproducibility

`scripts/package_final.py` renders the two PDFs, packages actual source/locks/migrations/native modules/models/docs/evidence, checks ZIP CRC and required members, excludes secrets/caches/real biometric data, and verifies PDF text extraction and page counts. `release/final/artifact-checks.json` contains sizes and SHA-256 hashes. Previous release artifacts are preserved. Generated Xcode/Gradle project folders are reproduced from source configuration. APKs are distributed separately from the source ZIP; the genuine debug artifact and its checksum/build evidence are committed on the completion branch. No IPA is fabricated.

Exact continuation checkpoint

1. On an Apple Silicon Mac run the documented simulator build, resolve/review CocoaPods lockfile and compile the Swift modules. Fix any native errors and commit the genuine results.

2. On physical Android and iPhone hardware validate enrollment, head-turn orientation, blink timing, precise/reduced GPS, current-network/scan unavailability, API retries, key loss/reinstall/reset and attendance/faculty history end to end. Record actual failures, OS/device versions and timing.

3. Configure a genuine Apple identity/root/profile and validate certificate/assertion interoperability, replay/counters and unsupported devices. Complete Play Integrity's authenticated Google provider adapter and physical-device tests; never replace it with client booleans.

4. Run consented calibration and presentation/injection evaluation; report measured uncertainty and fairness rather than invented accuracy. Establish accessible review alternatives.

5. Resolve the two upstream tooling advisories when compatible fixes exist; finish operational TLS, least-privilege DB access, secrets rotation, retention, backups and load testing before institutional deployment.

Local build procedures, EAS/TestFlight instructions and troubleshooting are in SETUP_AND_DEMO_GUIDE.md and CROSS_PLATFORM_IMPLEMENTATION.md. If GitHub publishing is unavailable, the local branch and verified artifacts remain intact; run `git push -u origin spvs/final-completion-cross-platform` and open a PR into `spvs/complete-implementation-20261022` without force-pushing.

References and provenance

The following are implementation references, not claims of deployment or experimentally measured accuracy. Consult the linked versioned source and official documentation when reproducing the project.

1. FastAPI documentation: https://fastapi.tiangolo.com/ — dependency injection, validation, security and API documentation.

2. SQLAlchemy 2.0 documentation: https://docs.sqlalchemy.org/en/20/ — ORM mappings, sessions and transactions.

3. PostgreSQL 16 documentation: https://www.postgresql.org/docs/16/ — constraints, indexes and row-level locking.

4. Alembic documentation: https://alembic.sqlalchemy.org/ — migration generation and application.

5. OpenCV Zoo, pinned commit: https://github.com/opencv/opencv_zoo/tree/48b10fa505aeb51fbd127fc60a5c660040e823b7 — official YuNet/SFace assets, Git LFS object hashes and licenses.

6. OpenCV DNN face tutorial: https://docs.opencv.org/4.x/d0/dd4/tutorial_dnn_face.html — detection, aligned SFace features and matching metrics. Its example threshold is not a local SPVS calibration result.

7. MediaPipe Face Landmarker guide: https://ai.google.dev/edge/mediapipe/solutions/vision/face_landmarker — Tasks API and official model bundle.

8. MediaPipe source/license: https://github.com/google-ai-edge/mediapipe — Apache 2.0 code and model-related resources.

9. Argon2-cffi documentation: https://argon2-cffi.readthedocs.io/ — maintained password hashing interface.

10. Cryptography Fernet documentation: https://cryptography.io/en/latest/fernet/ — authenticated symmetric encryption and key handling.

11. Expo documentation: https://docs.expo.dev/ — development builds, Camera, Location, SecureStore and local modules.

12. Android Wi-Fi scanning overview: https://developer.android.com/develop/connectivity/wifi/wifi-scan — scan restrictions, permissions and throttling.

13. Android Play Integrity overview: https://developer.android.com/google/play/integrity/overview — external provider integration requirements.

14. Apple App Attest: https://developer.apple.com/documentation/devicecheck/establishing-your-app-s-integrity — attestation boundary not configured in this distribution.

15. React, Vite and TanStack Query: https://react.dev/ ; https://vite.dev/ ; https://tanstack.com/query/latest — client architecture.

16. Playwright and pytest: https://playwright.dev/ ; https://docs.pytest.org/ — browser and backend testing.

17. scikit-image astronaut sample, version 0.24.0: https://github.com/scikit-image/scikit-image/blob/v0.24.0/skimage/data/astronaut.png — public NASA astronaut portrait used for model smoke tests.

18. matplotlib Grace Hopper sample, version 3.9.2: https://github.com/matplotlib/matplotlib/blob/v3.9.2/lib/matplotlib/mpl-data/sample_data/grace_hopper.jpg — public reference portrait used for a different-person smoke test.

19. npm advisory GHSA-vfj7-8cjw-p6xm: https://github.com/advisories/GHSA-vfj7-8cjw-p6xm — unresolved `braces` issue recorded by audit.

20. npm advisory GHSA-86w9-cpqp-85rv: https://github.com/advisories/GHSA-86w9-cpqp-85rv — unresolved `node-forge` issue recorded by audit.

License texts for the downloaded face detector and recognizer are included in `LICENSE-YUNET.txt` and `LICENSE-SFACE.txt`. The project source inherits the repository's MIT license. Third-party dependency licenses remain their respective authors' licenses. Reference portraits are downloaded only for local tests and are excluded from submission archives.